

Phase 3 : Nettoyage selon les 6 Piliers de la Qualité

1. Objectifs et Architecture

L'objectif de cette phase est de transformer les données brutes (couche **Bronze**) en données fiables et exploitables (couche **Silver**). Pour ce faire, nous avons développé un script en Python `03_Nettoyage.ipynb` qui se connecte à notre base de données MariaDB qui permet de nettoyer les données selon les règles techniques et métiers qui ont été étudiées dans les étapes précédentes.

Nous avons adopté une architecture stricte séparant deux types de traitements :

1. **Le Data Engineering (Règles Techniques)** : Garantir que la donnée est propre, normée et techniquement valide.
2. **La Logique Métier (Business Rules)** : Garantir que la donnée respecte les règles fonctionnelles du domaine médical et financier.

Cette approche nous permet de respecter les **6 piliers de la Gouvernance des Données** : Complétude, Unicité, Exactitude, Validité, Actualité et Cohérence.

Afin de garantir une automatisation robuste et sans erreur, nous avons implémenté une stratégie d'écriture **idempotente** vers MariaDB. En configurant l'export avec le paramètre `if_exists='replace'`, le script vérifie automatiquement l'état de la table `healthcare_silver` : si elle existe déjà, elle est supprimée et intégralement recréée avec les données nettoyées. Cette méthode "Drop & Create" empêche l'accumulation de doublons techniques lors de ré-exécutions successives des scripts et assure que la base de données contient toujours une version unique, propre et à jour, sans intervention manuelle.

2. Traitements Techniques (Data Quality)

- **Gestion de la Complétude (P1 - Critique)** :
Action : Suppression des enregistrements dont les clés primaires fonctionnelles (*Name*, *Date of Admission*) sont manquantes. Une admission "anonyme" ou "sans date" est techniquement inexploitable.
- **Standardisation et Exactitude (P3 - Mineur)** :

(Exécuté en amont pour maximiser l'efficacité du dédoublonnage)

Action : Normalisation des champs textuels (*Name, Doctor, Hospital...*) via l'application du format "Title Case" et le nettoyage des espaces superflus (*trim*). Cela permet d'harmoniser les saisies (ex: "david" devient "David").

Action : Nettoyage des noms d'hôpitaux via des expressions régulières (*Regex*) pour supprimer les suffixes juridiques parasites ("LLC", "Inc", "Group") qui nuisaient à la lisibilité et créaient de faux doublons.

- **Garantie d'Unicité (P1 - Critique) :**

Action : Dédoublonnage strict **post-nettoyage**. Grâce à la standardisation préalable, nous avons pu identifier et supprimer efficacement les lignes où un même patient apparaissait plusieurs fois (ex: "David Smith" vs "david smith"), garantissant ainsi la fiabilité des volumes d'activité.

- **Validité des Données (P2 - Majeur) :**

Action : Bornage des valeurs numériques (*Sanity Check*). Les âges impossibles (négatifs ou > 120 ans) ont été remplacés par *NaN* pour ne pas biaiser les calculs statistiques.

Action : Vérification des listes de référence. Les groupes sanguins hors nomenclature ont été standardisés en "Unknown".

```
# T01 : Completude (Suppression si cles manquantes)
init_len = len(df_clean)
df_clean = df_clean.dropna(subset=['Name', 'Date of Admission'])
print(f"    -> [T01] Lignes incomplètes supprimées : {init_len - len(df_clean)}")

# T03 : Exactitude (Standardisation Title Case + Trim)
text_cols = ['Name', 'Doctor', 'Hospital', 'Medical Condition', 'Medication', 'Insurance Provider']
for col in text_cols:
    df_clean[col] = df_clean[col].str.title().str.strip()
print(f"    -> [T03] Textes standardisés (Title Case).")

# T04 : Exactitude (Nettoyage Hopitaux)
df_clean['Hospital'] = df_clean['Hospital'].apply(lambda x: re.sub(r'(?i)\s+(LLC|Inc|Ltd|PLC|Group)$', '', str(x)))
print(f"    -> [T04] Noms d'hopitaux nettoyés.")

# --- MAINTENANT ON DÉDOUBLONNE (T02) ---

# T02 : Unicité (Dédoublonnage strict)
init_len = len(df_clean)
df_clean = df_clean.drop_duplicates(subset=['Name', 'Date of Admission'], keep='first')
print(f"    -> [T02] Doublons techniques supprimés : {init_len - len(df_clean)}")

# T05 : Validité (Bornage Age)
mask_age = (df_clean['Age'] < 0) | (df_clean['Age'] > 120)
df_clean.loc[mask_age, 'Age'] = np.nan
print(f"    -> [T05] Ages aberrants mis à NaN : {mask_age.sum()}")

# T06 : Validité (Conformité Listes - Groupe Sanguin)
valid_blood = ['A+', 'A-', 'B+', 'B-', 'O+', 'O-', 'AB+', 'AB-']
mask_blood = ~df_clean['Blood Type'].isin(valid_blood)
df_clean.loc[mask_blood, 'Blood Type'] = 'Unknown'
print(f"    -> [T06] Groupes sanguins invalides corrigés : {mask_blood.sum()}")
```

```
[PHASE TECHNIQUE] Nettoyage Structurel...
-> [T01] Lignes incomplètes supprimées : 0
-> [T03] Textes standardisés (Title Case).
-> [T04] Noms d'hôpitaux nettoyés.
-> [T02] Doublons techniques supprimés : 5514
-> [T05] Âges aberrants mis à NaN : 0
-> [T06] Groupes sanguins invalides corrigés : 0
```

3. Application des Règles Métier (Business Logic)

Ces règles assurent la cohérence fonctionnelle des données.

- **Fiabilité Financière :**
 - + *Règle* : Redressement des factures négatives en valeurs absolues.
 - + *Règle* : **Flagging** des anomalies de facturation (ex: séjours très courts facturés > 20k\$ ou séjours longs facturés < 500\$) pour audit ultérieur.
- **Cohérence Temporelle (Actualité) :**
 - + *Règle* : Suppression des séjours incohérents où la date de sortie est antérieure à la date d'entrée.
 - + *Règle* : Suppression des dates d'admission situées dans le futur.
- **Logique Assurantielle :**
 - + *Règle* : Correction automatique (Imputation) des patients de moins de 65 ans déclarés sous "Medicare", réassignés à "Blue Cross" pour refléter la réalité du marché.
- **Prudence Éthique sur les Données Cliniques :**
 - + Contrairement aux erreurs techniques, les incohérences médicales (ex: *Asthme traité par Pénicilline* ou *Diabète avec Test Normal*) ne sont pas supprimées pour conserver l'historique du dossier patient.
 - + *Action* : Création de colonnes d'alerte (Alert_Medication, Alert_Test) permettant d'isoler ces dossiers dans le Dashboard sans altérer la donnée source brute.

```

print("\n[PHASE METIER] Logique Fonctionnelle...")

# M01 : Finance (Facturation Positive)
neg_bills = (df_clean['Billing Amount'] < 0).sum()
df_clean['Billing Amount'] = df_clean['Billing Amount'].abs()
print(f"    -> [M01] Factures negatives redressees : {neg_bills}")

# Conversion Dates
df_clean['Date of Admission'] = pd.to_datetime(df_clean['Date of Admission'], errors='coerce')
df_clean['Discharge Date'] = pd.to_datetime(df_clean['Discharge Date'], errors='coerce')

# M02 : Actualite (Chronologie)
mask_chrono = df_clean['Discharge Date'] >= df_clean['Date of Admission']
invalid_dates = (~mask_chrono).sum()
df_clean = df_clean[mask_chrono]
print(f"    -> [M02] Incoherences chronologiques supprimees : {invalid_dates}")

# M03 : Actualite (Pas de dates futures)
mask_future = df_clean['Date of Admission'] > pd.Timestamp.now()
df_clean = df_clean[~mask_future]
print(f"    -> [M03] Admissions futures supprimees : {mask_future.sum()}")

# M04 : Coherence (Assurance vs Age)
mask_medicare = (df_clean['Age'] < 65) & (df_clean['Insurance Provider'] == 'Medicare')
df_clean.loc[mask_medicare, 'Insurance Provider'] = 'Blue Cross'
print(f"    -> [M04] Correction Medicare (<65 ans) : {mask_medicare.sum()}")

# M07 : Coherence (Facture vs Sejour)
df_clean['Length_of_Stay'] = (df_clean['Discharge Date'] - df_clean['Date of Admission']).dt.days
mask_bill_suspicious = (
    ((df_clean['Length_of_Stay'] <= 1) & (df_clean['Billing Amount'] > 20000)) |

```

```

    df_clean['Alert_Billing_Anomaly'] = mask_bill_suspicious
    print(f"    -> [M07] Anomalies Facture/Sejour flaggees : {mask_bill_suspicious.sum()}")

# M05 & M06 : Flagging Clinique (Medicaments & Tests)
# M06 Tests
mask_test = (df_clean['Medical Condition'] == 'Diabetes') & (df_clean['Test Results'] == 'Normal')
df_clean['Alert_Test'] = mask_test

# M05 Medicaments
incompatible_meds = {
    'Asthma': ['Penicillin', 'Lipitor'],
    'Cancer': ['Aspirin', 'Ibuprofen'],
    'Obesity': ['Penicillin', 'Aspirin']
}
correction_map = {'Asthma': 'Albuterol', 'Cancer': 'Chemo', 'Obesity': 'Diet'}

df_clean['Alert_Medication'] = False
df_clean['Medication_Corrected'] = df_clean['Medication']

count_meds = 0
for cond, meds in incompatible_meds.items():
    mask = (df_clean['Medical Condition'] == cond) & (df_clean['Medication'].isin(meds))
    if mask.sum() > 0:
        df_clean.loc[mask, 'Alert_Medication'] = True
        df_clean.loc[mask, 'Medication_Corrected'] = correction_map.get(cond, 'Other')
        count_meds += mask.sum()

print(f"    -> [M05/M06] Incoherences Cliniques flaggees : {count_meds + mask_test.sum()}")

```

```
[PHASE METIER] Logique Fonctionnelle...
-> [M01] Factures negatives redressees : 96
-> [M02] Incoherences chronologiques supprimees : 0
-> [M03] Admissions futures supprimees : 0
-> [M04] Correction Medicare (<65 ans) : 6983
-> [M07] Anomalies Facture/Sejour flaggees : 982
-> [M05/M06] Incoherences Cliniques flaggees : 12787
```

4. Bilan Technique et Stockage

Le script **03_Nettoyage.ipynb** exécute ces transformations de manière séquentielle.

- **Volume Initial (Bronze)** : 55 500 lignes.
- **Volume Final (Silver)** : 49986 lignes
- **Stockage** : Les données nettoyées ont été sauvegardées dans la table *healthcare_silver* sur MariaDB.

```
[EXPORT] Sauvegarde vers la couche Silver...
[SUCCESS] Table 'healthcare_silver' creee avec succes.
Volume Final : 49986 lignes.
Le dataset est pret pour le Dashboard.
```

Cette table **Silver** est désormais certifiée "propre" et servira de source unique de vérité pour la validation automatisée (Phase 4 avec Great Expectations).

Vérification de la présence de données dans la table *healthcare_silver* :

```
MariaDB [data_quality_db] select * from healthcare_silver limit 5;
```

Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	Doctor	Hospital	Insurance Provider	Billing Amount
Bobby Jackson	30	Male	B-	Cancer	2024-01-31 00:00:00	Matthew Smith	Sons And Miller	Blue Cross	18956.
281305978155	328	Urgent		2024-02-02 00:00:00	Paracetamol	Normal	2	0	0
0									
Leslie Terry	62	Male	A+	Obesity	2019-08-20 00:00:00	Samantha Davies	Kim	Blue Cross	33643.
327286577885	265	Emergency		2019-08-26 00:00:00	Ibuprofen	Inconclusive	6	0	0